

מכונת כדורים

מדינה (Madina) המציאה מכונת כדורים כדי לבדר את מתחרי ה-IOI. המבנה הפנימי של המכונה הוא כדלהלן:

- המכונה בנויה מ- N צמתים, הממוספרים מ-0 עד $N - 1$. צומת $N - 1$ נקרא **השורש**.
- עבור כל צומת u המקיים $0 \leq u < N - 1$, האב של הצומת u הוא צומת $P[u]$ המקיים $P[u] > u$, והצומת u הוא ילד של $P[u]$. לשורש אין אב. לצומת בלי ילדים קוראים **עלה**.

אתם לא יודעים את N או את האב $P[u]$ של כל צומת u . במקום, מדינה מספרת לך את M , מספר העלים, ושםספרי העלים הם $0, 1, \dots, M - 1$. המטרה שלך זה למצוא את המבנה הפנימי של המכונה באמצעות תפעול שלה.

כל צומת של המכונה יכול להכיל רק עד **כדור** אחד, ולכל כדור יש **ערך** שלם אי-שלילי. אנו אומרים שלצומת יש ערך x אם הוא מכיל כדור בעל ערך x . צומת נחשב **תפוס** אם הוא מכיל כדור; אחרת, הוא נחשב **פנוי**. ראשית, כל הצמתים פנויים.

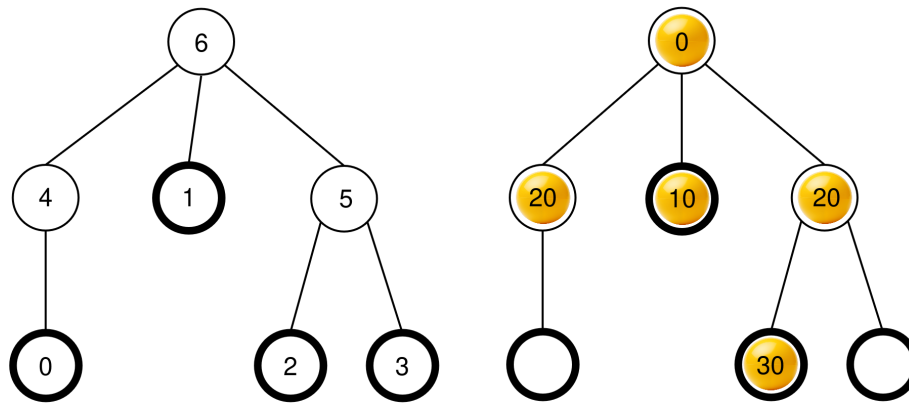
אתם יכולים לבצע שני סוגים של פעולות:

- **insert(U, X)**: נסיון להכניס כדור חדש בעל ערך X בעלה U .
 - אם העלה U תפוס, הפעולה תחזיר `false` בלי להכניס את הכדור.
 - אם העלה U פנוי, הכדור יונח בצומת U . לאחר מכן הכדור שוב ושוב יזוז לאב של הצומת הנוכחי שלו כל עוד האב הזה פנוי. התזוזה עוצרת כאשר הכדור מגיע לשורש או לצומת שאביו תפוס. הפעולה תחזיר `true`.
- **collect()**: אם השורש פנוי (מה שאומר שהמכונה ריקה), `collect` תחזיר מערך ריק. אחרת, הפעולה הרקורסיבית `traverse` המתוארת להלן תאסוף את הערכים של כל הכדורים שכרגע במכונה לתוך המערך S . ראשית, המערך S ריק ו-`traverse(N - 1)` תיקרא.

```
traverse(u):
    append the value of the ball at node u to the end of S
    let c[u] be the list of the occupied children of u
    sort c[u] in non-decreasing order according to the values
      of the balls at those nodes (if multiple nodes have the
      same value, they can appear in any order)
    for each v in c[u]:
        traverse(v)
```

אחרי שהפעולה הזאת מסתיימת, **כל הכדורים נמחקים מהמכונה**, ו-`collect` מחזירה S .

לדוגמה, חשבו על מכונה עם $N = 7$ צמתים ומערך האבות $P = [4, 6, 5, 5, 6, 6]$. התמונה משמאל מציגה את המכונה הריקה עם מספרי הצמתים, בזמן שהתמונה מימין מציגה תצורה אפשרית של כדורים אחרי רצף כלשהו של פעולות `insert` (הרצף נתון בחלק של הדוגמה).



כש- `collect()` נקראת, השורש (צומת 6) תפוס, אז S מאותחל כ- $S = []$ ו- `traverse(6)` נקראת.

`traverse(6)`: לצומת 6 יש ערך 0; הוספה שלו ל- S תתן $S = [0]$. הילדים של צומת 6 הם 4, 1 ו-5, אשר כולם תפוסים. הערכים שלהם הם 10, 20 ו-20 בהתאמה. מיון בסדר לא יורד מניב $c[6] = [1, 5, 4]$ (שימו לב ש- $c[6] = [1, 4, 5]$ יהיה חוקי גם כן, כי הצמתים 4 ו-5 בעלי אותו ערך). הפעולה אחר כך קוראת ל-`traverse` עבור כל צומת ב- $c[6]$ בסדר זה.

- **`traverse(1)`:** לצומת 1 יש ערך 10; הוספתו ל- S תוביל ל- $S = [0, 10]$. לצומת 1 אין בנים תפוסים, אז הקריאה הזאת מסתיימת.

- **`traverse(5)`:** לצומת 5 ערך 20; הוספתו ל- S תוביל ל- $S = [0, 10, 20]$. לצומת 5 יש צומת תפוס אחד, הצומת 2, אז $c[5] = [2]$ והפעולה קוראת ל- `traverse(2)`.

- **`traverse(2)`:** לצומת 2 ערך 30; הוספתו ל- S תוביל ל- $S = [0, 10, 20, 30]$. לצומת 2 אין ילדים תפוסים, אז הקריאה הזאת נגמרת.
- ההרצה חוזרת ל- `traverse(5)` שעובד את כל הילדים ב- $c[5]$, אז הקריאה עבורו נגמרת.

- **`traverse(4)`:** לצומת 4 ערך 20; הוספתו ל- S תוביל ל- $S = [0, 10, 20, 30, 20]$. לצומת 4 אין עוד ילדים תפוסים, אז הקריאה הזו נגמרת.

כל הקריאות הסתיימו ואין יותר צמתים לעבד. לבסוף, כל כדור נמחק מהמכונה ו-`collect` מחזירה את המערך $S = [0, 10, 20, 30, 20]$.

המשימה שלכם היא להבין את מספר הצמתים N ולדווח מערך של אבות $R = [R[0], R[1], \dots, R[N-2]]$ שמתאר את מבנה המכונה, אך אולי עם מספור שונה של הצמתים שהם לא העלים או השורש. פורמלית, דיווח על מערך R נחשב לנכון אם ניתן להקצות תווית יחודית $L[u]$ ($0 \leq L[u] < N$) לכל צומת u במכונה כך ש:

- $L[u] = u$ עבור כל $0 \leq u < M$ ועבור $u = N-1$, ובנוסף
- $L[P[u]] = R[L[u]]$ עבור כל $0 \leq u < N-1$.

לא נדרש שיתקיים $R[u] > u$. המכונה חייבת להיות ריקה שאתם מדווחים את הפתרון שלכם.

יהי K מספר הקריאות ל-`collect` שבוצעו, ו- B הערך המירבי של כדור על פני כל הקריאות ל-`insert`. יהי $C = K + B$. אז C חייב לא לחרוג מ-1000. הניקוד שלכם בתת משימות מסוימות תלוי בערך של C .

פרטי מימוש

עליכם לממש את הפונקציה הבאה.

```
std::vector<int> find_structure(int M)
```

- M : מספר העלים במכונה.
- הפונקציה תיקרא בדיוק פעם אחת עבור כל מקרה בדיקה.
- על פונקציה זו להחזיר מערך $R = [R[0], R[1], \dots, R[N-2]]$ באורך $N-1$, מערך אבות נכון.

כדי לתקשר עם המכונה, הפונקציה שלכם יכולה לקרוא לשתי הפונקציות הבאות.

הפונקציה הראשונה היא:

```
bool insert(int U, int X)
```

- U : האינדקס של העלה שבו הכדור צריך להיות מוכנס. התנאי $0 \leq U < M$ חייב להתקיים.
- X : הערך השלם של הכדור. התנאי $0 \leq X \leq 1000$ חייב להתקיים.
- פונקציה זו מחזירה `true` אם הכדור הוכנס בהצלחה, או `false` אם העלה U כבר היה תפוס.
- פונקציה זו יכולה להקרא לכל היותר 500 000 פעמים.

הפונקציה השנייה היא:

```
std::vector<int> collect()
```

- אוספת את הערכים של כל הכדורים שהוכנסו, מרוקנת את המכונה, ומחזירה את המערך שנאסף S .

אם בנקודה כלשהי במהלך הרצת התוכנית שלכם C חורג מ-1000, הפתרון שלכם יקבל את המשוב `Output isn't correct: Too many resources used`.

מבנה המכונה מקובע לפני ש-`find_structure` נקראת. הגריידר **דטרמיניסטי** במובן שאם אתם מריצים אותו פעמיים ושתי הריצות מבצעות את אותו רצף פעולות, הקריאות ל-`collect` יחזירו את אותם המערכים.

אילוצים

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

תתי משימות

תת משימה	ניקוד	אילוצים נוספים
1	5	לשורש יש בדיוק M ילדים.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	ללא אילוצים נוספים.

בתתי משימות 3 ו-4, הניקוד שלכם תלוי בערך של C כדלקמן.

תת משימה 3 (25 נקודות)

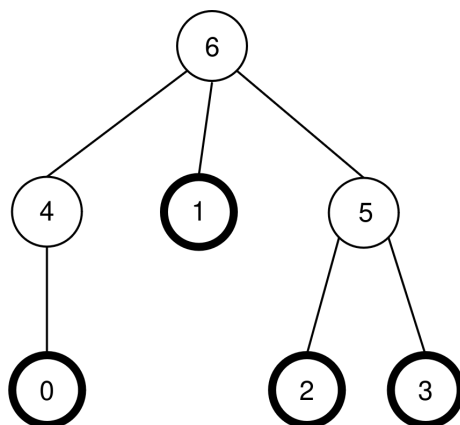
ניקוד	חסמים
0	$1000 < C$
13	$45 < C \leq 1000$
25	$C \leq 45$

תת משימה 4 (60 נקודות)

ניקוד	חסמים
0	$1000 < C$
7	$200 < C \leq 1000$
$47 - \frac{C}{5}$	$71 < C \leq 200$
$104 - C$	$44 < C \leq 71$
60	$C \leq 44$

דוגמה

חישבו על אותה המכונה שבתיאור המשימה, אז $N = 7$, $M = 4$, ו- $P = [4, 6, 5, 5, 6, 6]$. המכונה מוצגת שוב באיור הבא:



הגרייטר מבצע את הקריאות הבאות:

```
find_structure(4)
```

הפונקציה יכולה לבצע את רצף הקריאות הבא:

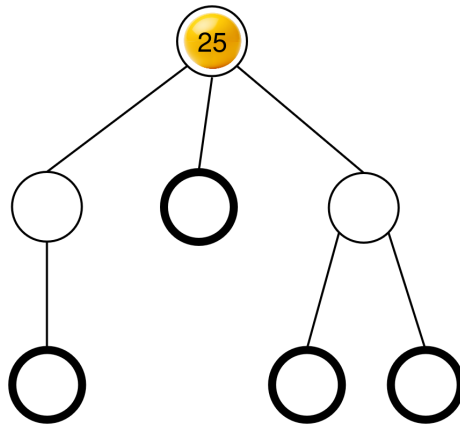
1. $\text{insert}(0, 0)$: עלה 0 פנוי, אז כדור עם הערך 0 מונח בעלה 0. צומת $P[0] = 4$ פנוי, אז הכדור זז לצומת 4. צומת $P[4] = 6$ פנוי, אז הכדור זז לצומת 6. כיוון ש-6 הוא השורש, התנועה מפסיקה. הפונקציה מחזירה `true`.
2. $\text{insert}(3, 20)$: עלה 3 פנוי, אז כדור עם הערך 20 מונח בעלה 3. צומת $P[3] = 5$ פנוי, אז הכדור זז לצומת 5. מכיוון ש- $P[5] = 6$ תפוס, התנועה מפסיקה. הפונקציה מחזירה `true`.
3. $\text{insert}(1, 10)$: עלה 1 פנוי, אז כדור עם הערך 10 מונח בעלה 1. משום ש- $P[1] = 6$ תפוס, הכדור נשאר בצומת 1. הפונקציה מחזירה `true`.
4. $\text{insert}(2, 30)$: עלה 2 פנוי, אז כדור עם הערך 30 מונח בעלה 2. משום ש- $P[2] = 5$ תפוס, הכדור נשאר בצומת 2. הפונקציה מחזירה `true`.
5. $\text{insert}(1, 25)$: עלה 1 תפוס, אז הכדור לא מונח בעלה 1. הפונקציה מחזירה `false`.
6. $\text{insert}(0, 20)$: עלה 0 פנוי, אז כדור עם הערך 20 מונח בעלה 0. צומת $P[0] = 4$ פנוי, אז הכדור זז לצומת 4. כיוון ש- $P[4] = 6$ תפוס, התנועה נעצרת. הפונקציה מחזירה `true`.

סידור זה של הכדורים הוצג כבר בתיאור המשימה. אחר כך, הפונקציה יכולה לבצע את הקריאה:

```
collect()
```

ההרצה של הקריאה הזו כבר הוסברה בתיאור המשימה, אז קריאה זו מחזירה $S = [0, 10, 20, 30, 20]$. לאחר מכן, המכונה שוב ריקה.

לאחר מכן, הפונקציה יכולה לבצע את הקריאה $\text{insert}(2, 25)$. העלה 2 פנוי, אז כדור עם הערך 25 מונח בעלה 0. הכדור הזה זז לאחר מכן לצומת 5, ואז לצומת 6, שם הוא נעצר. הקריאה מחזירה `true`. תצורת הכדורים שנוצרת מומחשת באיור הבא.



לבסוף, הפונקציה יכולה לבצע את הקריאה `collect()` שמחזירה $S = [25]$ ומרוקנת את המכונה.

קיימים שני מערכות אבות נכונים ש-`find_structure` יכולה להחזיר: $R = P = [4, 6, 5, 5, 6, 6]$ (שמתאים לסימון $(L = [0, 1, 2, 3, 4, 5, 6])$ ו- $R = [5, 6, 4, 4, 6, 6]$ (שמתאים לסימון $(L = [0, 1, 2, 3, 5, 4, 6])$. בדוגמה זו, $K = 2$ ו- $B = 30$, אז $C = 32$.

גריידר לדוגמה

פורמט קלט:

```

N M
P[0] P[1] P[2] ... P[N-2]

```

פורמט פלט:

```

K B C
Q
R[0] R[1] R[2] ... R[Q-1]

```

כאן, Q זה אורך המערך R המוחזר על ידי `find_structure`.